

Ingegneria del Software – ISW

State Diagram UML

Gli state diagram UML sono un modo visivo per descrivere come un sistema o oggetto cambia nel tempo in risposta a certi eventi o azioni.

- **Uno stato rappresenta una condizione o situazione** in cui si trova un oggetto o un sistema. es: “in attesa”, “in esecuzione”, “errore”, “spento”, ecc.
- **Una transizione è un cambiamento di stato** che avviene quando si verifica un evento (ad esempio un input, un segnale, un’azione dell’utente, ecc.).

Ogni stato è rappresentato da un rettangolo con angoli arrotondati. Dentro uno stato possiamo avere:

- entry → azione all’ingresso
- do → azione durante lo stato
- exit → azione all’uscita

Le transizioni sono rappresentate da frecce etichettate “evento [guardia] / azione”, dove:

- evento → l’input o segnale che causa la transizione
- [guardia] → una condizione booleana opzionale (deve essere vera per attivare la transizione)
- / azione → un’azione da eseguire durante la transizione

Lo stato iniziale (START) si disegna come un cerchio pieno (nero). È il punto da cui inizia il sistema.

Lo stato finale (END) si disegna come un cerchio con un altro cerchio dentro.

Per Esame: Esercizi in cui il prof ci dà **input x of A = output x di B** vanno “letti al contrario” ed interpretati come **B -- x --> A**. Cioè B va in A se riceve x

Esempio:

// Begin input/output connections for system components

Input 12 of SC2 = Output 12 of SC1

Input 21 of SC1 = Output 21 of SC2

Input 23 of SC3 = Output 23 of SC2

Input 24 of SC4 = Output 24 of SC2

Input 32 of SC2 = Output 32 of SC3

Input 43 of SC3 = Output 43 of SC4

// End input/output connections for system components

SC1 va in SC2 con 12

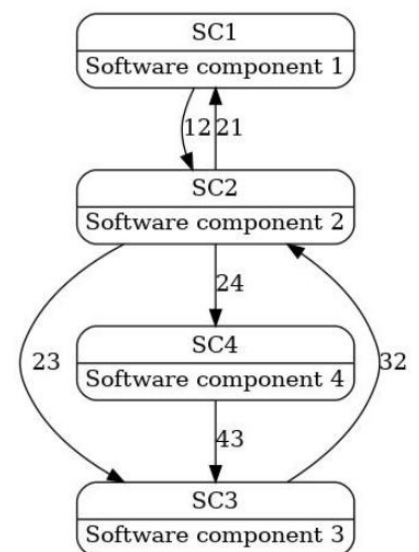
SC2 va in SC1 con 21

SC2 va in SC3 con 23

SC2 va in SC4 con 24

SC3 va in SC2 con 32

SC4 va in SC3 con 43

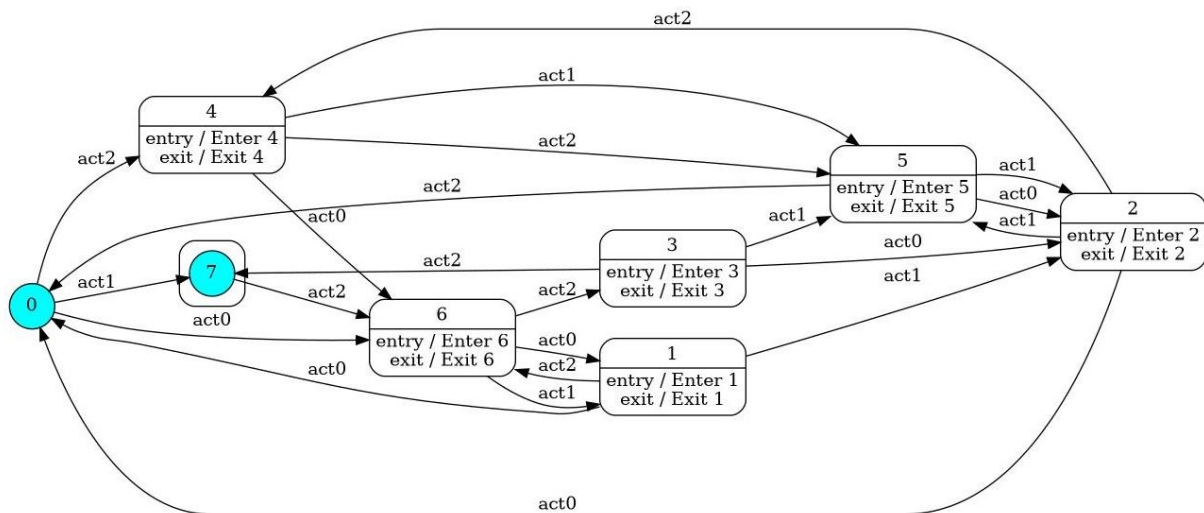


State Converge

Def La state coverage di un insieme di test cases (cioè sequenze di inputs) per uno state diagram è la percentuale di stati (inclusi START ed END) raggiunti almeno una volta.

Esercizio: Si consideri il seguente insieme di test cases:

- Test case 1: act2 act0 act2 act0 act0
- Test case 2: act1
- Test case 3: act0 act0 act2 act1



Quale è la stima della state coverage per i test cases di cui sopra?

Soluzione: Tracciando i tre test case sul diagramma (seguendo le etichette delle transizioni):

- **Test 1:** act2 act0 act2 act0 act0 — visita gli stati 0 → 4 → 6 → 3 → 2 → ... (quindi raggiunge 0,4,6,3,2).
- **Test 2:** act1 — 0 → 7 (raggiunge 0 e 7).
- **Test 3:** act0 act0 act2 act1 — 0 → 6 → 1 → 6 → 2 (raggiunge 0,6,1,2).

Unendo i risultati otteniamo gli stati raggiunti: **{0,1,2,3,4,6,7}** — cioè **7 stati su 8** (lo stato 5 non viene coperto da questi test).

Transition Converge

Def La transition coverage di un insieme di test cases (cioè sequenze di inputs) per uno state diagram è la percentuale di transizioni (archi nel grafo dello state diagram) percorsi almeno una volta.

Branch Converge

Def Il **branch coverage** di un insieme di test cases è la percentuale di branch del programma che sono attraversati da almeno un test case.

Path Coverage

Def La **path coverage** di un insieme di test cases è la percentuale di percorsi (sequenze di istruzioni fino al return) che vengono eseguiti almeno una volta nel programma.

Catene di Markov

Def Una catena di Markov è un modello matematico per descrivere un sistema che evolve nel tempo passando da uno stato a un altro in modo probabilistico.

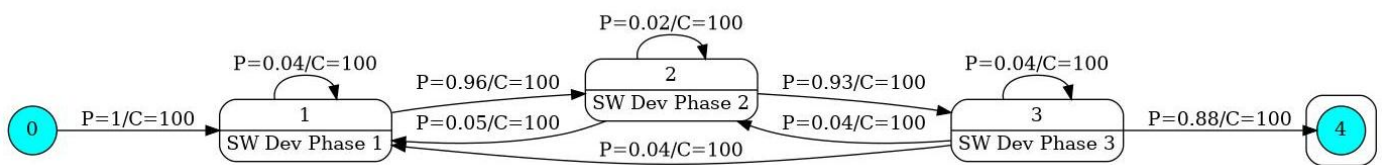
Nel nostro caso, gli stati rappresentano fasi dello sviluppo software (es. Fase 1, Fase 2, Fase 3, Fine), e le transizioni rappresentano i possibili passaggi da una fase all'altra, con una certa probabilità e un costo associato.

Rappresentazione: Ogni processo è rappresentato da un **grafo orientato**, i cui archi sono etichettati con:

- P = probabilità di transizione
- C = costo associato alla transizione

Tramite questa rappresentazione è molto semplice calcolare la probabilità che si verifichi un determinato percorso. Per farlo è sufficiente prendere le probabilità delle singole transizioni e moltiplicarle tra loro.

Esempio: Supponiamo di avere la seguente catena di Markov:



Vogliamo calcolare la probabilità che si verifichi il percorso 1-2-2-3-4. Per farlo ci basta prendere le singole probabilità delle transizioni e moltiplicarle tra loro. Otterremo quindi:

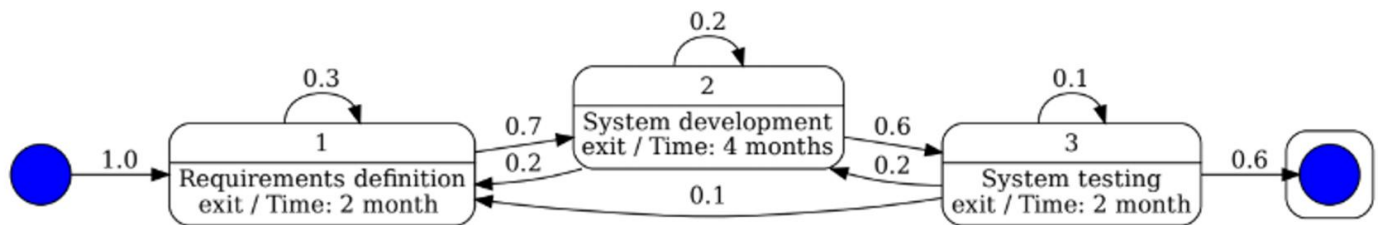
$$P_{\text{tot}} = 0.96 \times 0.02 \times 0.93 \times 0.88 = 0.0157...$$

Calcolo del Costo di Completamento

Per calcolare la probabilità che un processo termini con un **costo di esattamente x** (o esattamente in un tempo x) è sufficiente prendere tutti i percorsi, che portano il processo a terminare, e che spendono esattamente x. Una volta trovi questi percorsi:

- Calcoliamo le probabilità dei singoli percorsi
- Sommare tra loro le probabilità

Esempio: Supponiamo di avere il seguente caso e voler calcolare la probabilità che il processo termini in esattamente 10 mesi.



I percorsi che ci permettono di terminare in 10 mesi sono:

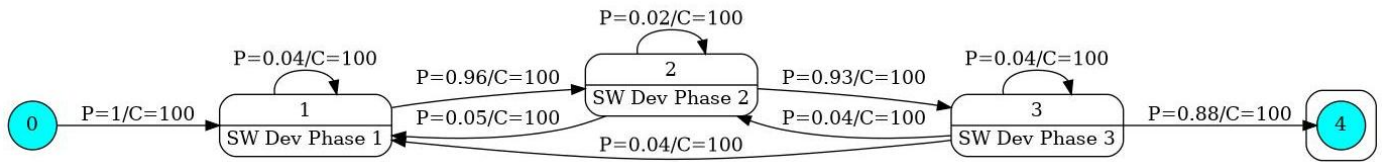
- Percorso1 : $0 \rightarrow 1 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4$ $\Rightarrow P_1 = 1 \times 0.3 \times 0.7 \times 0.6 \times 0.6 = 0.07560$
- Percorso2 : $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 3 \rightarrow 4$ $\Rightarrow P_2 = 1 \times 0.7 \times 0.6 \times 0.1 \times 0.6 = 0.02520$

La probabilità per ciascun percorso viene calcolata moltiplicando le probabilità di transizione lungo il percorso.

Sommiamo ora le probabilità dei singoli percorsi per ottenere la probabilità totale che il processo termini in esattamente 10 mesi:

$$P_{\text{tot}} = P_1 + P_2 = 0.07560 + 0.02520 = 0.1008$$

Esempio 2: Data la seguente catena di Markov, calcolare la probabilità che il processo completi con un costo di esattamente 400.0000



(Come prima) i percorsi che ci permettono di terminare con costo esattamente 400 sono:

- Percorso 1: $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \Rightarrow P_1 = 1 \times 0.96 \times 0.93 \times 0.88 = 0.76032$

Poiché questo è l'unico percorso che ci permette di terminare con costo 400 il risultato sarà $P_{\text{tot}} = P_1 = 0.76032$

Precondizioni

Def Sono le condizioni che devono essere vere prima dell'esecuzione di una funzione (o di un blocco di codice). Servono a garantire che i parametri o lo stato iniziale del programma siano validi.

```
int divide(int a, int b) {  
    assert(b != 0); // ✅ pre-condizione: non si può dividere per 0  
    return a / b;  
}
```

Se chi chiama la funzione passa $b = 0$, l'assert fallisce subito, segnalando un errore prima che accada qualcosa di sbagliato (come una divisione per zero).

Invarianti

Def Sono le condizioni che devono essere sempre vere durante l'esecuzione di un certo blocco di codice. Tipicamente in ogni iterazione di un ciclo o in ogni punto interno di una funzione.

```
int f(int x, int y) {  
    int z = x;  
    int w = y;  
  
    // invariante: la somma di z e w deve essere < 1 oppure > 7  
    assert( (z + w < 1) || (z + w > 7) );  
  
    // ... corpo della funzione ...  
  
    assert( (z + w < 1) || (z + w > 7) ); // può essere ricontrollato alla fine  
    return z + w;  
}
```

In questo caso, l'assert controlla una proprietà che deve sempre restare vera durante l'esecuzione della funzione.

Precondizioni

Def Sono le condizioni che devono essere vere alla fine dell'esecuzione di una funzione. Servono per verificare che il risultato sia corretto.

```
int square(int x) {  
    int result = x * x;  
    assert(result >= 0); // ✅ post-condizione: il quadrato non può essere negativo  
    return result;  
}
```

assert()

Nel linguaggio C, `assert()` è una macro (non una funzione vera e propria) usata per verificare che una certa condizione sia vera durante l'esecuzione di un programma. La sintassi è **`assert(expression);`**

- Se **expression** è vera ($\neq 0$) non succede nulla, il programma continua normalmente.
- Se **expression** è falsa ($= 0$), stampa un messaggio d'errore su `stderr` e termina il programma chiamando `abort()`.

Software Testing

Il **software testing** è il processo che si occupa di verificare e validare che un programma faccia ciò che deve fare, senza errori, e in modo conforme ai requisiti specificati. In pratica:

- **Verifica:** controllare che il software costruito soddisfi i requisiti del sistema.
- **Validazione:** controllare che il software costruito soddisfi le necessità dell'utente.

Poiché il testing è una fase fondamentale dello sviluppo di un software esistono moltissime forme diverse di testing. Le classificazioni principali includono:

- **Testing Statico:** Si fa senza eseguire codice. Serve per scoprire errori nella struttura o nella logica prima dell'esecuzione.
- **Testing Dinamico:** Si fa eseguendo il programma. Serve per vedere se il software si comporta correttamente. Si può a sua volta dividere in livelli, che seguono lo sviluppo del software:
 - *Unit Testing:* Fatto dagli sviluppatori per testare singole unità di codice
 - *Integration Testing:* Fatto dagli sviluppatori e dal team Quality Assurance (QA) per verificare che i vari moduli funzionino bene insieme
 - *System Testing:* Fatto dal Team QA per testare il sistema completo
 - *Acceptance Testing:* Fatto dai Clienti e Utenti Finali per verificare che il Software soddisfi i requisiti reali.
- **Alpha Testing:** è una fase di verifica interna del software, eseguita prima che il prodotto venga rilasciato ai clienti o agli utenti esterni. È il primo test completo del sistema, condotto dai programmatori e dal team interno. L'obiettivo dell'alpha testing è scoprire e correggere il maggior numero possibile di errori, malfunzionamenti o incoerenze, prima che il software sia visto da utenti esterni o da clienti in una fase pubblica (che sarà la beta).
- **Beta Testing:** Il software viene rilasciato a utenti esterni selezionati o a un pubblico più ampio. LO scopo principale è testare il software in condizioni reali, con hardware, configurazioni e comportamenti degli utenti reali per raccogliere feedback su usabilità, funzionalità e stabilità.

Unit Testing

Lo unit testing è una attività di verifica del software che mira a testare le unità di codice più piccole (in genere funzioni, metodi o moduli) in modo isolato, per assicurarsi che ciascuna si comporti come previsto, rispetti i requisiti funzionali, e gestisca correttamente gli input e gli errori.

Si tratta del **primo livello di test nel ciclo di sviluppo** software (prima di integration test, system test e acceptance test). Nel processo di unit testing si distinguono quattro fasi o task principali:

1. **TDEF — Definition of Test Cases:** Consiste nel definire i casi di test, cioè le situazioni (input → output atteso) con cui si vuole verificare il comportamento della funzione. Ogni caso di test deve avere un input e un risultato atteso. Talvolta si aggiungono anche condizioni iniziali e finali note. Dei buoni casi di test dovrebbero coprire situazioni normali (funzionamento tipico), casi limite (boundary conditions) e situazioni anomale (errori, input non validi).
2. **TORC — Definition of Test Oracle:** Il test oracle è il meccanismo che permette di sapere se il risultato ottenuto dal test è corretto o meno. Può essere un valore atteso definito manualmente oppure una funzione che calcola automaticamente l'output corretto. Senza un oracle, non puoi sapere se un test è "passato" o "fallito".
3. **TEXE — Execution of Test Cases:** È la fase in cui i test vengono realmente eseguiti sulla funzione o modulo. Si confronta l'output prodotto dal programma con quello definito dal test oracle. Questa attività può essere manuale o automatizzata tramite framework (es. JUnit in Java, pytest in Python, CppUnit in C++).
4. **TVAL — Evaluation of Test Results:** Si analizzano i risultati dei test per determinare quali test sono passati/falliti, la gravità e la causa degli errori. Include anche la documentazione dei risultati e l'eventuale revisione dei test.

Per Esame: L'ordine delle fasi segue lo schema TVAL < TEXE, TEXE < TDEF, TVAL < TORC, dove A < B indica che B viene eseguito PRIMA di A.

Test-Driven Development (TDD)

Il Test-Driven Development (TDD) è una metodologia di sviluppo software in cui i test guidano la scrittura del codice. Una delle caratteristiche fondamentali di questa tecnica è che “**TDD inizia sempre dai test**”. Il ciclo di TDD è chiamato spesso Red-Green-Refactor:

- **Red:** Scrivi un test automatico per la nuova funzionalità o comportamento desiderato. Eseguendo il test questo sicuramente fallirà perché la funzionalità non è ancora implementata.
- **Green:** Scrivi il codice minimo necessario per far passare il test.
- **Refactor:** Migliora il codice senza modificare il comportamento testato. Mantieni tutti i test verdi per assicurarti che il refactoring non rompa nulla.

Esempio: Supponiamo di voler implementare una funzione che calcola il massimo tra due numeri:

1. Red: scriviamo i test

```
assert(max(5, 10) == 10);  
assert(max(-2, 3) == 3);  
assert(max(7, 7) == 7);
```

2. Green: implemento il minimo codice per far passare i test

```
int max(int a, int b) {  
    if (a > b)  
        return a;  
    else  
        return b;  
}
```

3. Refactor: il codice è già semplice, ma se fosse più complesso, lo migliorerei senza rompere il test

```
int max(int a, int b) {  
    return (a > b) ? a : b;  
}
```


Calcolo del Rischio

Il rischio R può essere calcolato come $R = P \cdot C$, dove P è la probabilità dell'evento avverso (software failure nel nostro contesto) e C è il costo dell'occorrenza dell'evento avverso. Assumiamo che la probabilità P sia legata al costo di sviluppo S dalla formula $P = e^{-b \cdot S}$, dove b è una opportuna costante note da dati storici aziendali. Da tutta questa premessa possiamo cogliere le seguenti formule per risolvere gli esercizi:

Variabile da calcolare	Formula
P	$P = R / C$
S	$S = (1/b) \cdot \ln(C / R)$
C	$C = R \cdot e^{(bS)}$
b	$b = (1/S) \cdot \ln(C / R)$
R	$R = C \cdot e^{(-bS)}$
S (da P)	$S = -(1/b) \cdot \ln(P)$

Metodi Agile

I metodi Agile nascono alla fine degli anni '90 come reazione ai limiti percepiti dei modelli di sviluppo tradizionali, in particolare del modello a cascata (Waterfall). Quest'ultimo imponeva una sequenza rigida di fasi — analisi, progettazione, implementazione, test, manutenzione — che spesso portava a risultati poco flessibili, ritardi, costi elevati e prodotti finali non sempre in linea con le reali esigenze del cliente.

In un contesto di mercato sempre più dinamico, dove i requisiti possono cambiare rapidamente, i professionisti del software iniziarono a cercare approcci più adattivi e iterativi, in grado di rispondere al cambiamento in modo rapido e continuo.

Manifesto Agile

Il momento fondante del movimento Agile è la pubblicazione, nel 2001, del Manifesto Agile, firmato da 17 esperti del settore. Questo documento non proponeva un singolo metodo, ma definiva un insieme di valori e principi comuni a diverse metodologie già esistenti o emergenti. Il Manifesto Agile si basa su quattro valori principali:

1. **Individui e Interazioni più che Processi e Strumenti.** Si riconosce che le persone, la collaborazione e la comunicazione diretta sono più importanti di una mera aderenza a procedure o strumenti formali.
2. **Software funzionante più che Documentazione Esaustiva.** La priorità è consegnare un prodotto che funzioni e generi valore per il cliente, piuttosto che produrre grandi quantità di documentazione teorica.
3. **Collaborazione col cliente più che Negoziazione dei Contratti.** Il cliente è parte integrante del processo di sviluppo: non solo fornisce requisiti iniziali, ma partecipa attivamente con feedback continui e adattamenti.
4. **Rispondere al Cambiamento più che Seguire un Piano.** L'agilità consiste nella capacità di modificare direzione quando necessario, accettando il cambiamento come parte naturale del processo.

Oltre ai quattro valori, il Manifesto definisce dodici principi operativi che guidano lo sviluppo Agile. Essi enfatizzano concetti come:

- la consegna continua di software funzionante
- l'accoglienza dei cambiamenti anche in fasi avanzate del progetto
- la collaborazione costante tra sviluppatori e clienti
- la motivazione e autonomia dei team
- la comunicazione diretta come mezzo più efficace
- l'attenzione alla qualità tecnica e al design semplice
- la riflessione periodica sul processo per migliorarlo continuamente

Questi principi rappresentano il cuore culturale dell'approccio Agile: più che un insieme di regole fisse, Agile è una filosofia di lavoro centrata sull'adattabilità e sul valore.

Caratteristiche del Metodo Agile

Lo sviluppo Agile è iterativo e incrementale. Invece di pianificare e sviluppare l'intero sistema in un'unica lunga sequenza, il progetto viene suddiviso in piccoli incrementi funzionali (spesso detti *sprint*, *iteration* o *release*). Ogni iterazione produce una versione del software realmente eseguibile, che può essere testata, valutata e migliorata attraverso il feedback dell'utente.

Ciò permette di:

- ridurre il rischio complessivo del progetto
- maggiore flessibilità rispetto ai cambiamenti di requisiti.
- evidenziare problemi e necessità di cambiamento in anticipo
- mantenere il cliente costantemente coinvolto
- garantire una maggiore trasparenza sullo stato del lavoro

Tuttavia, Agile non è adatto a ogni contesto.

- Può risultare difficile da applicare in grandi organizzazioni molto gerarchiche.
- Richiede team esperti, collaborativi e disciplinati, capaci di autogestirsi.
- La documentazione ridotta può creare problemi in progetti ad alta complessità o in ambienti regolamentati (es. aerospaziale, sanitario).
- Il coinvolgimento continuo del cliente è essenziale, ma non sempre realisticamente disponibile.

Metodi Plain-Drive

Def Un approccio plain-drive è “**diretto al codice**”: prima si implementa, poi si corregge e si migliora, con poca burocrazia o formalismo.

Un approccio plain-drive punta direttamente al problema, senza complicazioni metodologiche. L'implementazione pratica viene prima di tutto: si scrive codice, si prova, si aggiusta, invece di passare molto tempo su diagrammi o documentazione complessa. I processi di design e analisi sono leggeri; la documentazione è essenziale, non eccessiva, e spesso si preferisce vedere subito un prototipo funzionante. In pratica, un approccio plain-drive può apparire così:

1. Comprendere a grandi linee il problema.
2. Scrivere codice per una soluzione base.
3. Testare rapidamente e correggere gli errori.
4. Iterare aggiungendo funzionalità o migliorando la qualità.

PRO: Rapidità di sviluppo, Facilità di comprensione, Flessibilità: facile cambiare approccio senza essere bloccati da un modello complesso.

CONTRO: Scalabilità limitata: in progetti grandi può diventare caotico. Mancanza di standardizzazione: difficile manutenzione a lungo termine. Documentazione minima.

Principali Approcci Plain-Drive

Waterfall: sviluppo lineare e sequenziale. Ogni fase deve essere completata prima di passare alla successiva. Fasi tipiche: Analisi dei requisiti → Progettazione (design) → Implementazione → Testing → Manutenzione

Incrementa /Iterative: il sistema viene sviluppato a piccoli blocchi (incrementi), ogni blocco è implementato, testato e rilasciato. A differenza del Waterfall le fasi si ripetono per ogni incremento, quindi più flessibile.

Prototyping: sviluppo rapido di una versione iniziale (“prototipo”) per chiarire i requisiti o testare idee. La sequenza tipica di fasi: progettazione rapida → implementazione prototipo → feedback → miglioramento.

Big Bang: si parte a scrivere direttamente il codice con poca pianificazione formale, e si prova tutto insieme alla fine.

Requirements Validation Activity

Check di Completezza: serve a verificare che l'insieme dei requisiti raccolti sia completo, cioè che copra tutte le funzionalità, i comportamenti e i vincoli richiesti dal sistema che si vuole sviluppare.

- Per esame: Assicurarsi che i requisiti descrivano tutte le funzionalità e vincoli (e.g., security, performance) del sistema desiderato dal customer.

Check di Validità: serve a verificare che i requisiti siano validi rispetto ai bisogni reali del cliente e degli utenti. Si controlla che se il sistema venisse costruito secondo quei requisiti, effettivamente risolverebbe il problema del cliente.

- Per Esame: Assicurarsi che un sistema che soddisfa i requisiti risolve il problema del "customer".

Check di Consistenza: Serve a verificare che non ci siano contraddizioni tra i requisiti raccolti. I requisiti devono poter coesistere logicamente e non imporsi vincoli opposti o incompatibili.

- Per Esame: Assicurarsi che non ci siano requisiti in conflitto con altri requisiti.

Check di Realismo: serve a verificare che i requisiti siano **realizzabili nella pratica**, cioè che la tecnologia disponibile sia in grado di supportarli, le risorse (budget, tempo, personale) siano sufficienti e che non siano presenti vincoli impossibili da soddisfare.

- Per Esame: Assicurarsi che, tenendo conto della tecnologia, budget e tempo disponibili, sia possibile realizzare un sistema che soddisfa i requisiti.

Verifiability: si riferisce alla possibilità di verificare ogni requisito individualmente, cioè dobbiamo poter stabilire se un requisito è stato soddisfatto o meno tramite test o misurazioni oggettive.

- Per Esame: Per ciascun requisito, dovremmo essere in grado di scrivere un insieme di test che può dimostrare che il sistema sviluppato soddisfa il requisito considerato.

Requisiti Funzionali e NON Funzionali

Un requisito funzionale descrive **cosa il sistema deve fare**, cioè le sue funzionalità o comportamenti specifici.

Un requisito non funzionale descrive **come il sistema deve fare**, cioè caratteristiche di qualità come prestazioni, affidabilità, sicurezza, usabilità, ecc.

- **Requisito di performance:** È un tipo di requisito non-funzionale, ma più specifico: riguarda le prestazioni misurabili del sistema (tempo, velocità, capacità, efficienza).

